

Doubly Linked List – Destroy List And Time Complexity

Program :

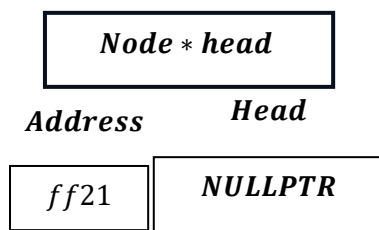
```
void destroyList()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}

...
case 12:
    destroyList();
    break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return ;
}
```

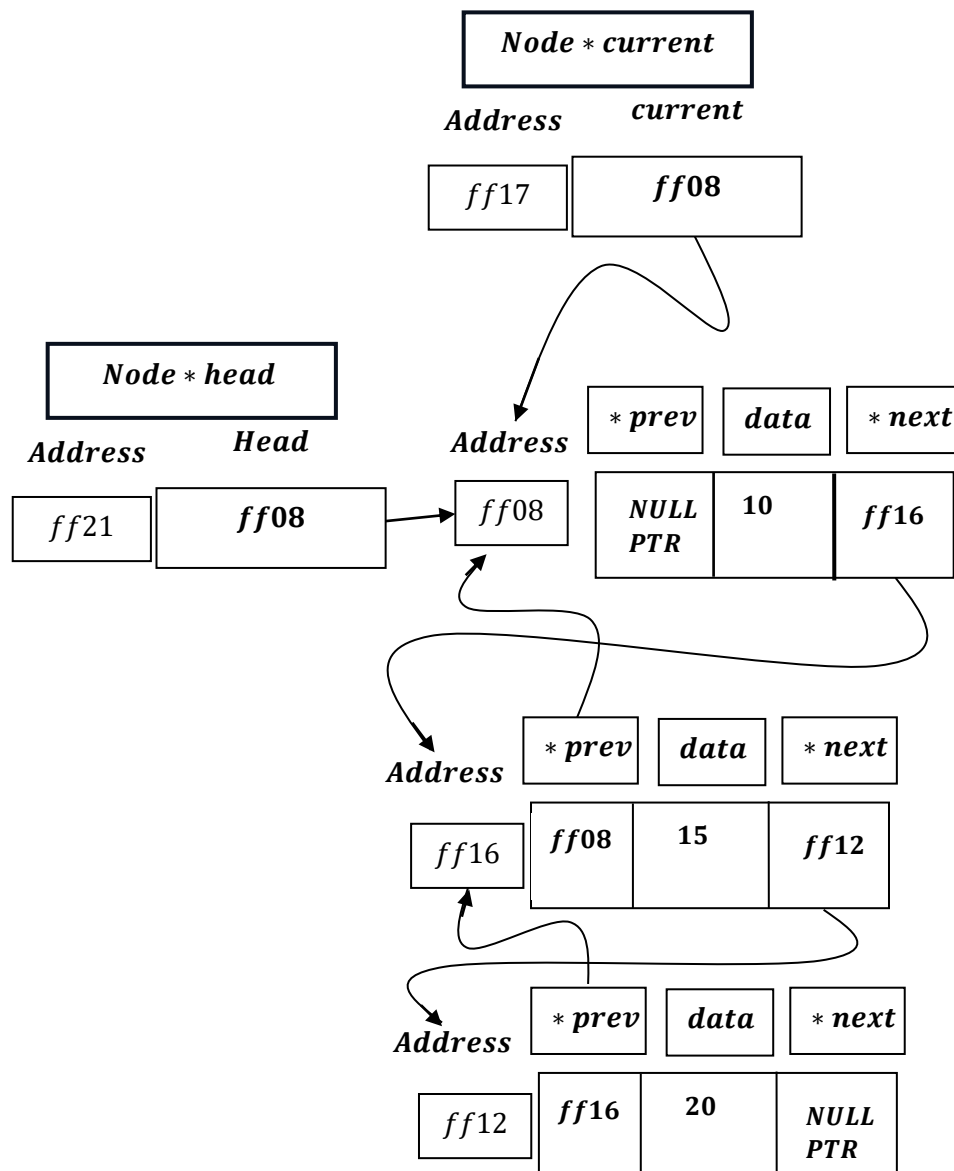


Then, it will print statement : `` List is empty.``

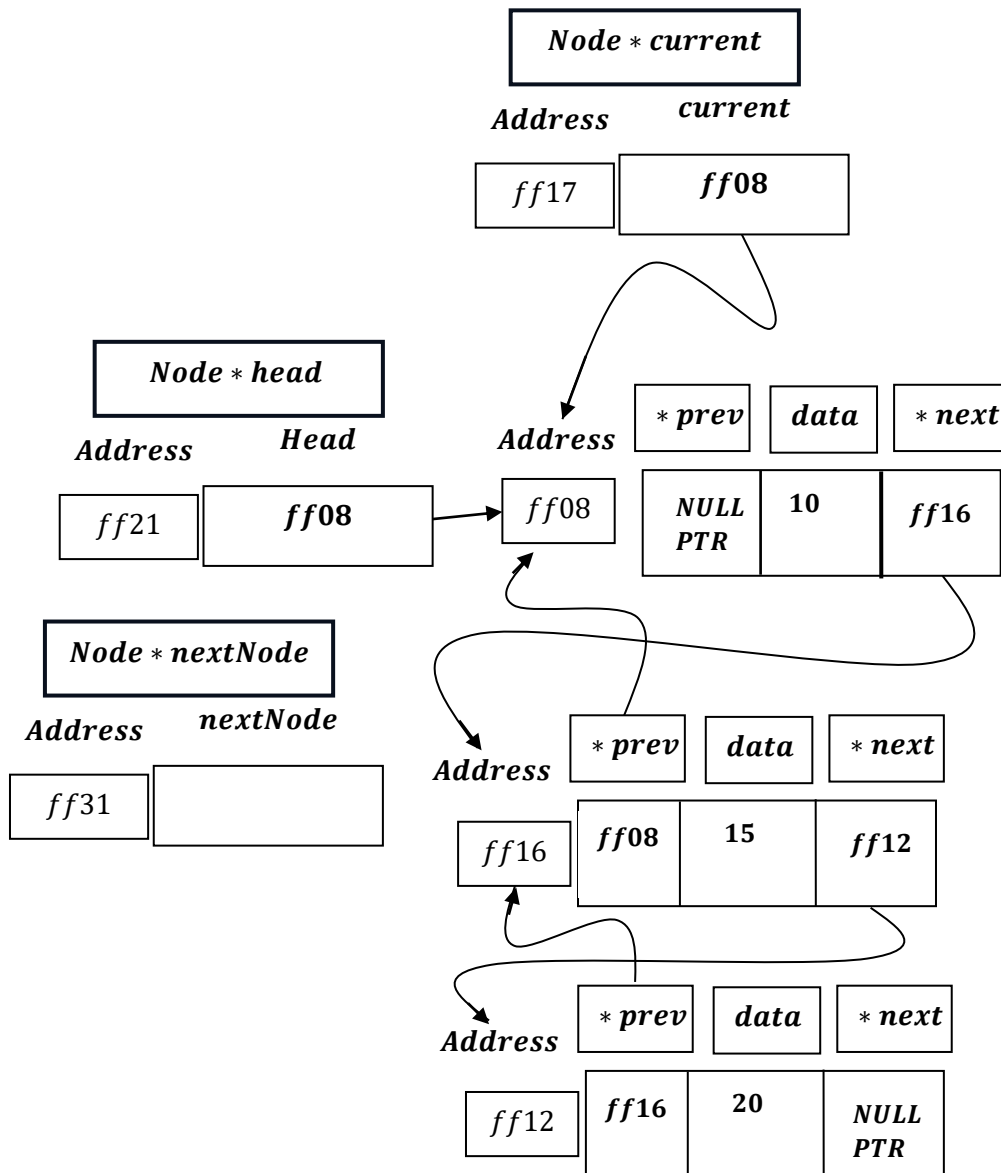
And, return statement will exit from the function.

B. When LIST is not Empty

```
Node *current = head;
```



```
Node *nextNode;
```



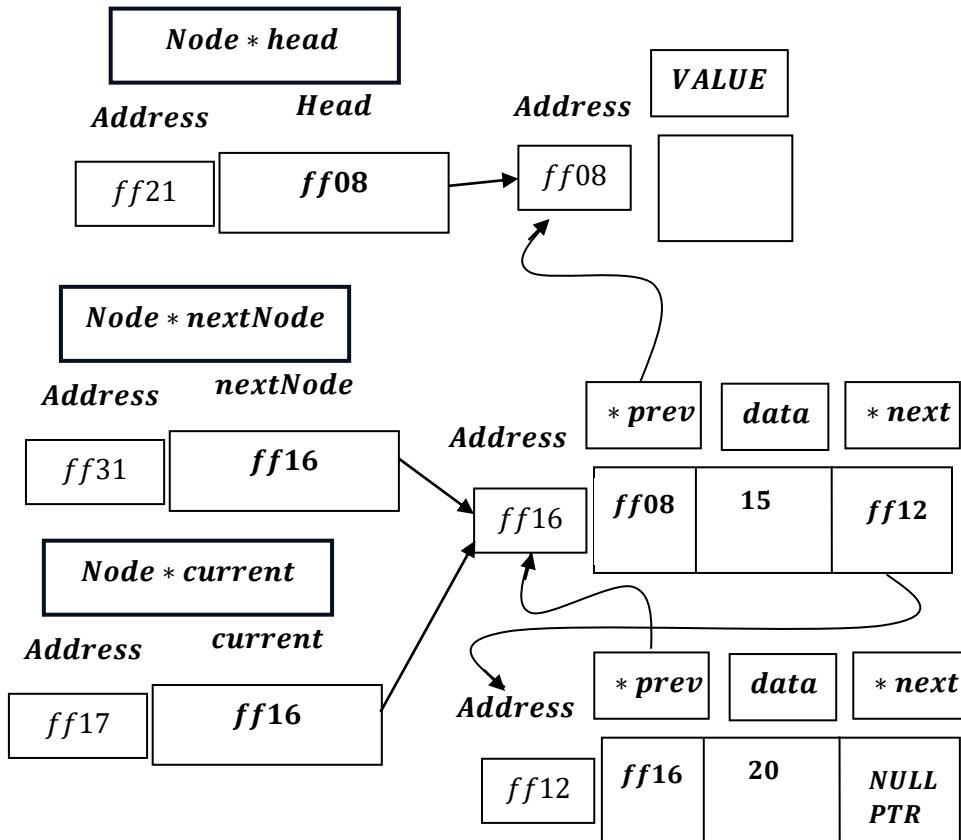
```
while (current != nullptr)
{
    nextNode = current->next;
    free(current);
    current = nextNode;
}
```

***current* $\neq$ NULLPTR is true ,hence:**

***nextNode* = *current* $\rightarrow$ *next* = *ff16*.**

free(current);

***current* = *nextNode* = *ff16*.**

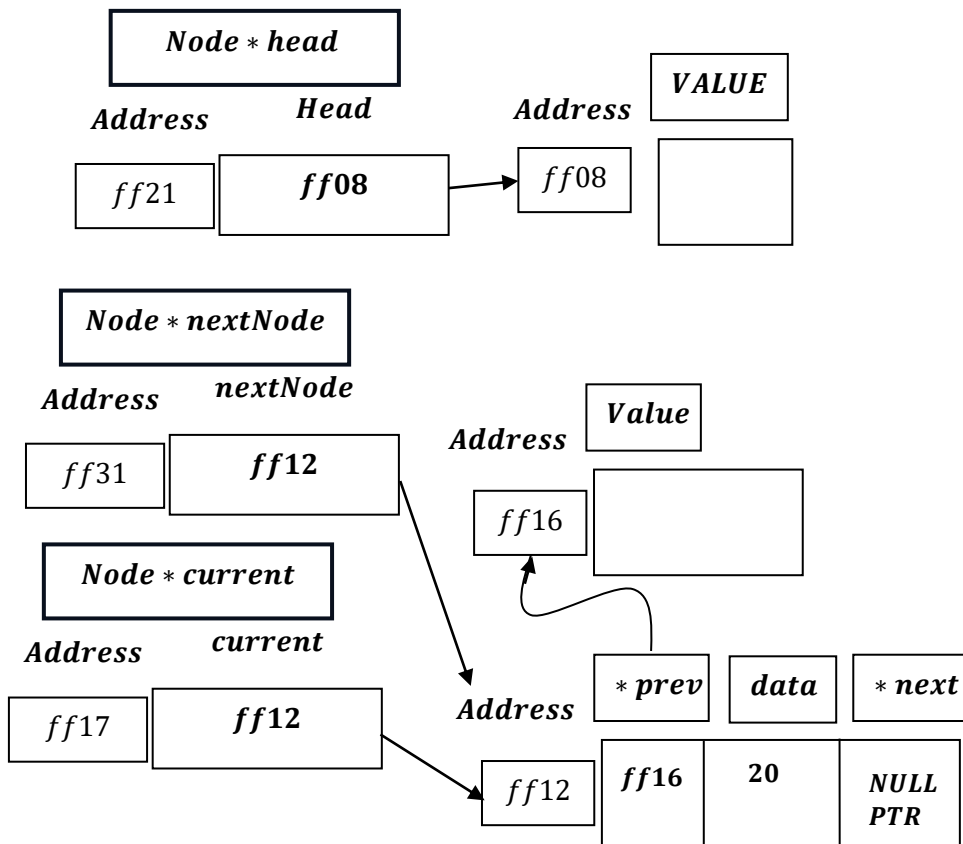


***current* $\neq$ NULLPTR is true ,hence:**

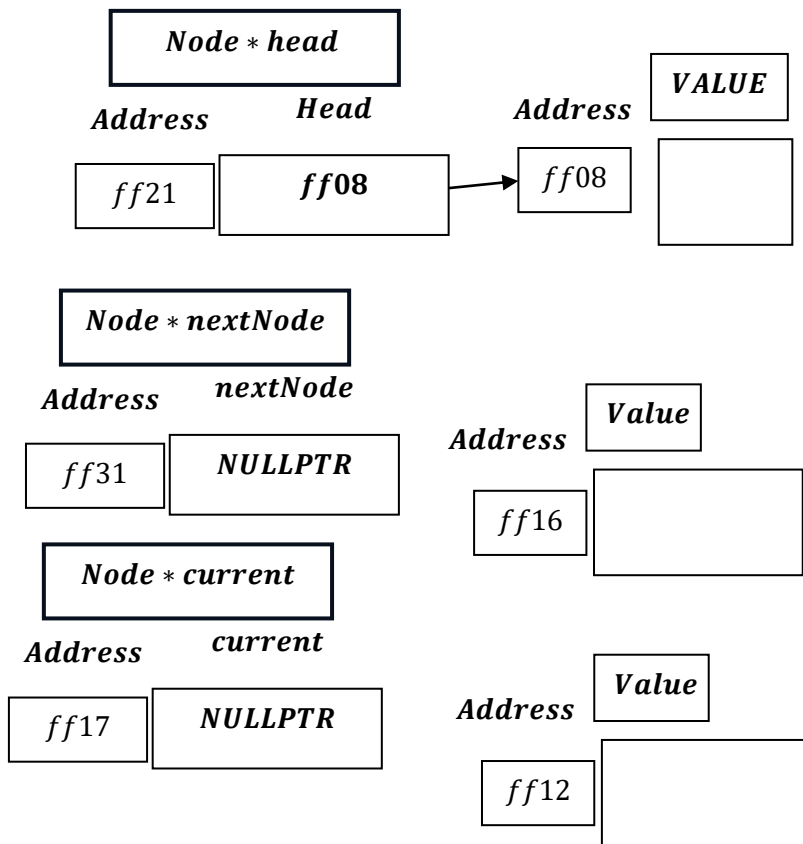
***nextNode* = *current* $\rightarrow$ *next* = *ff12*.**

free(current);

***current* = *nextNode* = *ff12*.**

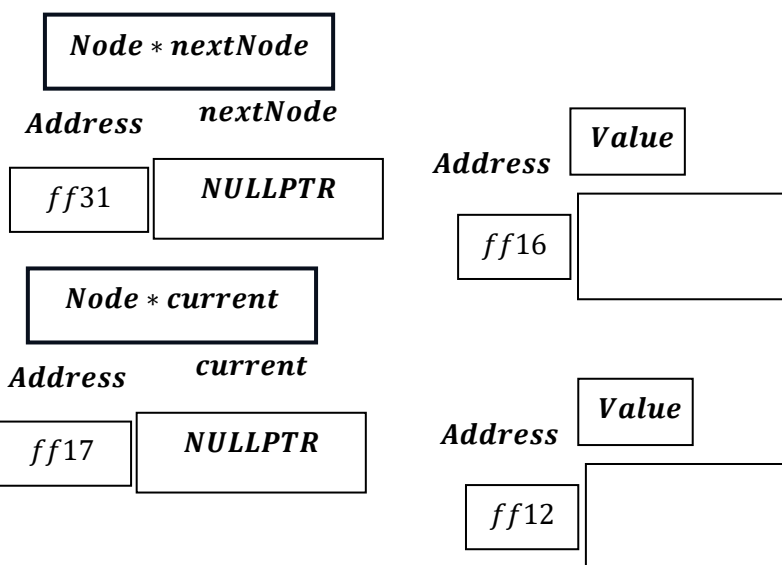
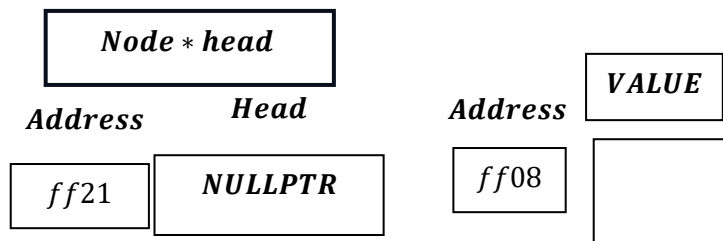


current $\neq$ *NULLPTR* is true , hence:
nextNode = *current* $\rightarrow$ *next* = *NULLPTR*.
free(current);
current = *nextNode* = *NULLPTR*.

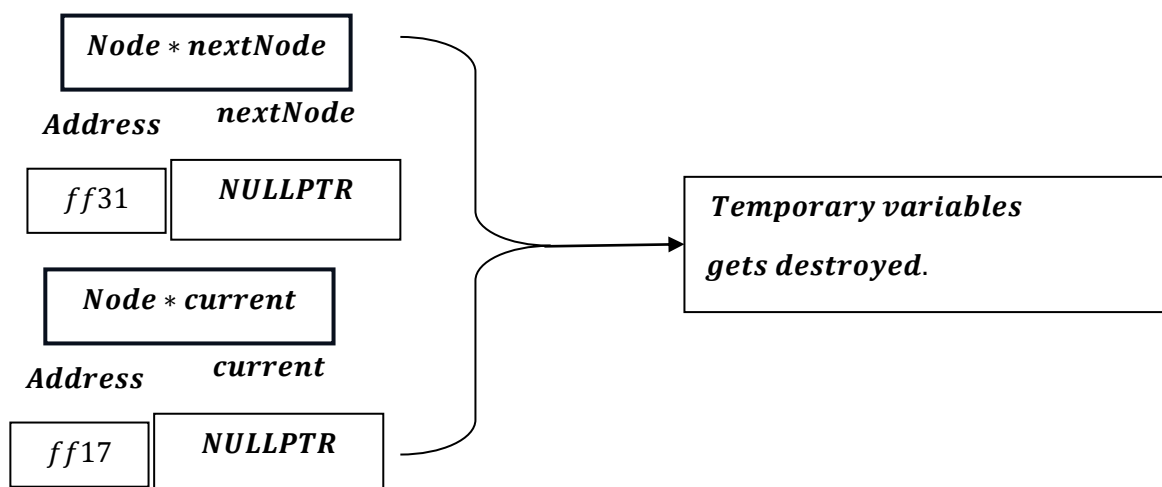


Now, $CURRENT = NULLPTR$, hence while ($TEMP \neq NULLPTR$) $\rightarrow$ condition fails.

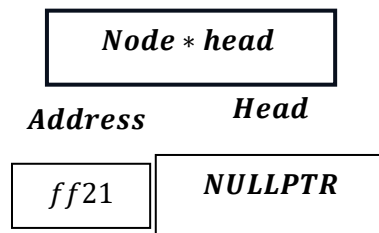
```
head = nullptr;
```



As function ends ,nextNode and current pointer, gets destroyed:



And we are left with:



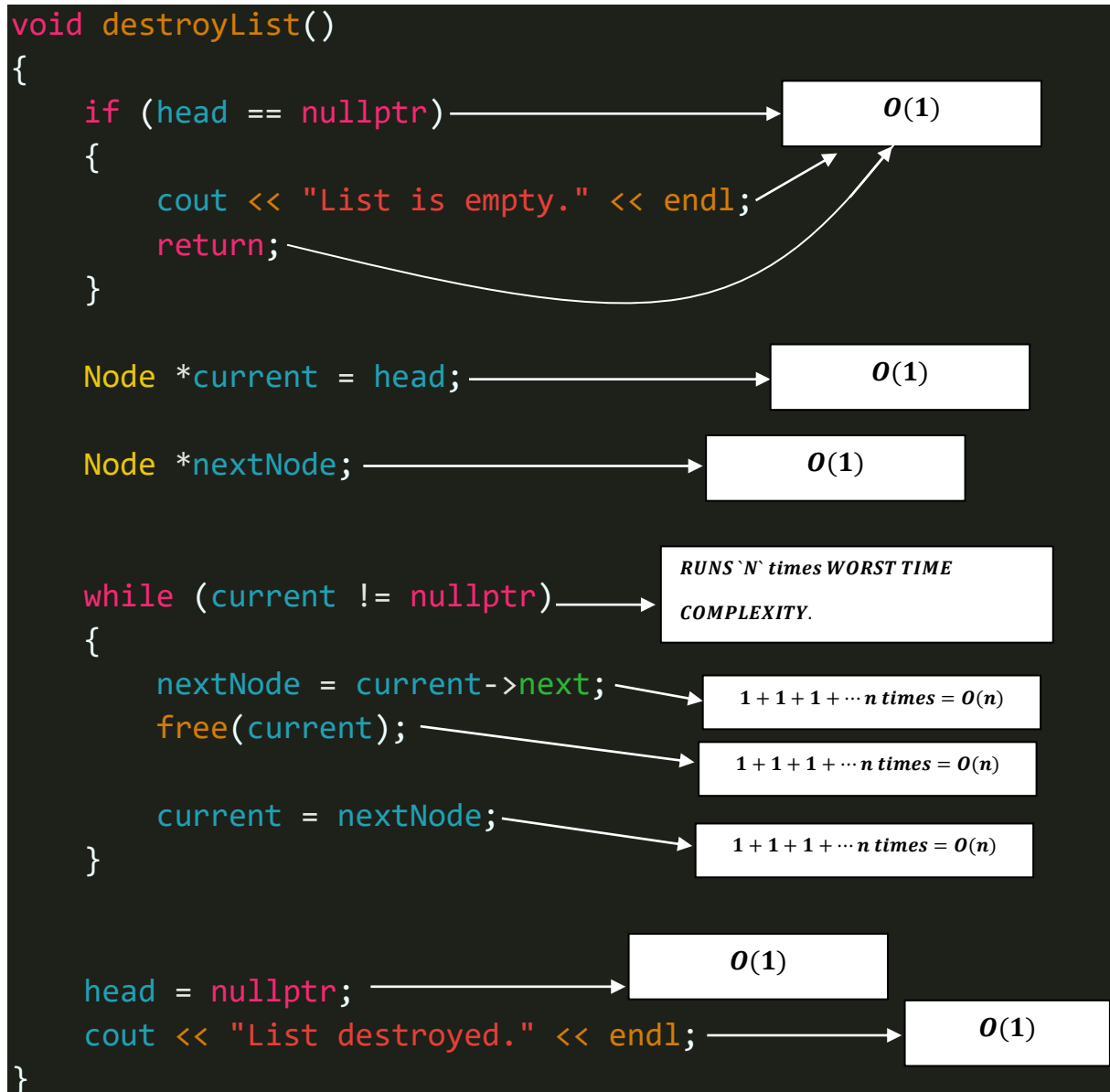
Doubly Linked List – Destroy List [Working Summary]:

This C++ function, `destroyList`, is designed to safely deallocate all the memory used by a linked list, effectively deleting it.

Working Summary

1. **Handles Empty List:** It first checks if the list is already empty (`head == nullptr`). If so, it prints a message and exits, as there's nothing to destroy.
 2. **Initializes Pointers:** If the list exists, it creates two pointers:
 - `current`: Starts at the `head` of the list.
 - `nextNode`: Is used to temporarily store the link to the next node.
 3. **Iterates and Deletes:** The function enters a `while` loop that continues as long as `current` is not `nullptr`. In each iteration:
 - It **saves the location** of the next node (`nextNode = current->next`) before deleting the current one. This is crucial to avoid losing the rest of the list.
 - It **frees the memory** occupied by the `current` node using `free(current)`.
 - It moves forward by setting `current` to the saved `nextNode`.
 4. **Resets Head Pointer:** After the loop has deleted every node, it sets the `head` pointer to `nullptr`. This is an important final step to ensure the list is officially marked as empty and prevents any dangling pointer issues.
 5. **Confirms Destruction:** Finally, it prints "List destroyed." to confirm the operation was completed.
-

Time Complexity Of Destroy List Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ ***Node * current = * head; this assignment takes constant time i.e. $O(1)$.***

→ ***Declaration of Node * nextNode ; takes constant time i.e. $O(1)$.***

→ ***while(current != nullptr) checking becomes true , then:***

→ ***nextNode = current → next; such assignment takes, $1 + 1 + \dots n$ times = n ;***

→ ***free(current) ; freeing memory pointed by current pointer;***

$1 + 1 + \dots n$ times = n ;

→ ***current = nextNode; such assignment takes, $1 + 1 + \dots n$ times = n ;***

Therefore ,total loop time complexity: $n + n + n = 3n = 3 \times O(n) = O(n)$.

→ ***head = nullptr; this assignment takes constant time i.e. $O(1)$.***

→ ***cout << "List destroyed."; takes constant time to execute i.e. $O(1)$.***

Hence total time complexity : $O(1) + O(1) + O(n) + O(1) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case).

There's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence , Destroy List function doesn't have a true average case.

Note ,by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Destroy List</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case (= Worst Case)	$\Theta(n)$
